

Comparing Traditional and Fractal Graphs Using Network Flow Analysis

Andrew Frantchouk

Abstract:

While the current U.S population growth rate is decreasing, the overall population is not. Factors such as migration, birth rates, and death rates are all influencing this growth, cascading into subsequent issues such as urban expansion. Urban expansion, sometimes also referred to as urbanization or urban sprawl, is the uncontrolled enlargement and creation of cities into standing natural areas. In 2023, about 86% of the U.S population resided in metro areas, along with a population increase of almost 3.2 million between 2023 and 2024. Urban expansion comes with significant social and environmental problems: increased pollution, loss of natural land/habitats, higher costs, and increased traffic congestion.

Overall, cities are complex systems which at a large scale, seem to behave and look almost like living organisms. The design of cities, and how they changed over time, are fascinating to look at. Interestingly enough, ancient cities seem to follow a more fractal-like pattern, surrounding a central hub, usually a building with a religious foundation. However, the main reason ancient cities evolved this way was to meet its inhabitants' needs within walking distance. This factor certainly favored pedestrians, whereas modern cities have evolved to focus on cars and other methods of transportation. Consequently, to conserve space and resources, our modern cities are grids.

By mirroring these ancient cities, and exploring the use of fractal generation, we could go into the future with both pedestrians, the environment, and the increasing population in mind. The project this report focuses on uses two graph generation techniques, where nodes represent buildings/intersections and edges represent roads/sidewalks. The first generation, or graph, is a traditional graph. Creating a loosely organized network, every node is connected with at least one edge. The second graph uses a recursive flower fractal structure, generating a more symmetrical and hopefully efficient topology.

After the feedback from Professor Guha, I shifted my efforts and attention strictly towards the network flow of both graphs. I tried my best not to mess with the generation of the graphs anymore, but simplified the traditional graph in hopes of reducing the amount of edges for a clearer graph. Regardless, after researching network flow diagrams/graphs, it was a challenge implementing all of its factors into my current project. However, after many iterations I was able to assign each edge (of both graphs) a random capacity, a source which was planted at the centermost of the network, and a sink which was the farthest distance from the sink.

My project aims to explore the efficiency and difference between the two graphs, mimicking cities to learn about the potential future of civil planning. Sometimes old ways work best, and perhaps the cities of the future will resemble the cities of the past.

Technical Introduction:

My project's magnum opus, and honestly the only thing that my project consists of, is my Python script. I've had some Python experience in the past, and it was quite refreshing to circle back to it after focusing solely on C and Java while at UCF. The program's heart is the NetworkX library, simulating and analyzing the two types of graph structures generated. The code generates both and evaluates these graphs with random edge capacities and a source to sink flow model.

The traditional graph is iteratively constructed with connecting nodes, guaranteeing every node is connected at least to one other. A few more additional edges are added to simulate different paths/roads and the organic growth of modern cities.

The fractal graph, on the other hand, is recursively generated, following a flower-like structure. Initially I aimed to create a H-Tree network, but could not get it to work. Therefore, I settled on a flower-like structure. There are so many different graphs with miniscule changes yet they go by different names which makes it incredibly hard to cite what I took inspiration from, but I digress. The flower graph is modeled after the, “Sierpinski’ weighted fractal network,” and the, “Cantor Dust weighted fractal network.” The recursive generation also messed with the node count, which made me implement a trim function, which cuts back on leaf nodes until the desired number of nodes are reached.

Once both graphs are generated, random capacities are assigned to the edges, an effort to simulate different road sizes, widths, speeds etc. There were many things I wanted to make work, but none I could. I struggled with coming up with an idea as to how to compare my graphs. I wanted more customization and had a few iterations where the user would input a desired source/sink node but the fractal graph would always run into some issues. I then tried to set the source node to the leftmost node and the sink to the rightmost node, however this would prove to be very troublesome and I couldn't go through with it. Eventually I settled on setting the source to the middle, node 0, and the sink to the farthest away node.

The core calculation metric comes from the Edmonds-Karp algorithm, used in the `maximum_flow` function from the NetworkX library. It calculates the maximum possible flow from the source to the sink while still respecting the edge capacity. Because of the, in my opinion, “unfair,” nature of the source location especially in the fractal graph, I added a normalized flow or average flow.

The graph visualization/drawing was handled and made with Matplotlib, which was a pain to install and properly set up. Nodes and edges are laid out in a spring layout algorithm, and edges affected by the flow calculation are color coded, red for edges at full capacity and green for partially used edges.

To summarize, the code creates two graphs and runs a network flow simulation on each one. It visualizes the architecture and, “traffic,” distribution. Aiming to provide insight to the secrets of city planning.

Prenote:

For clarity and cleanliness, I ended up showing the results with a generation of 10 vertices. However, the python code will be attached. Although, the graphs do get quite messy with more than 20 vertices I found. Which is why I'm limiting it to 10 for this report.

Basic Graph Results:

With an input of 10 vertices, the iterative random connection graph with guaranteed connectivity generates. A few more edges are added to simulate redundancy.

The source node in the attached image is 0. The sink was set to 7.

Image 1:

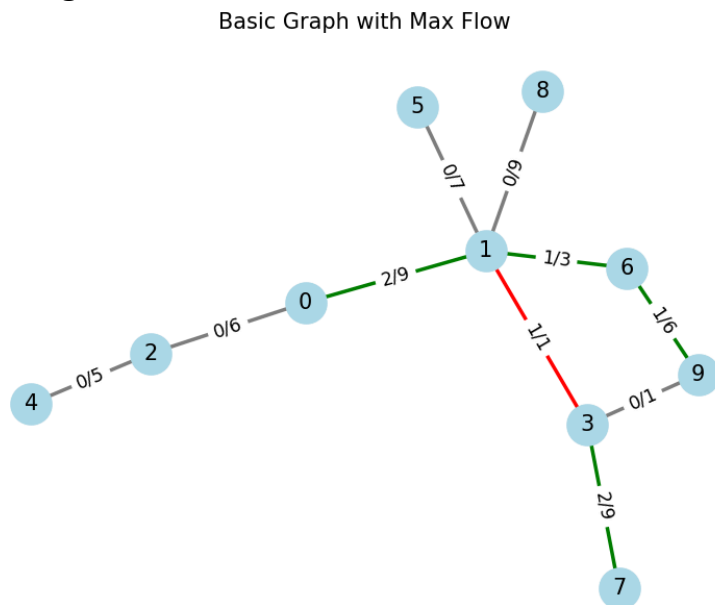


Image 2:

```
[Basic Graph] Source: 0, Sink: 7  
Max Flow: 2  
Normalized Flow: 2.00 / 56 = 0.036  
Active Edges: 6
```

Fractal Graph Results:

With an input of 10 vertices, the recursive fractal graph generates.
The source node in the attached image is 0. The sink was set to 22.

Image 1:

Fractal Graph with Max Flow

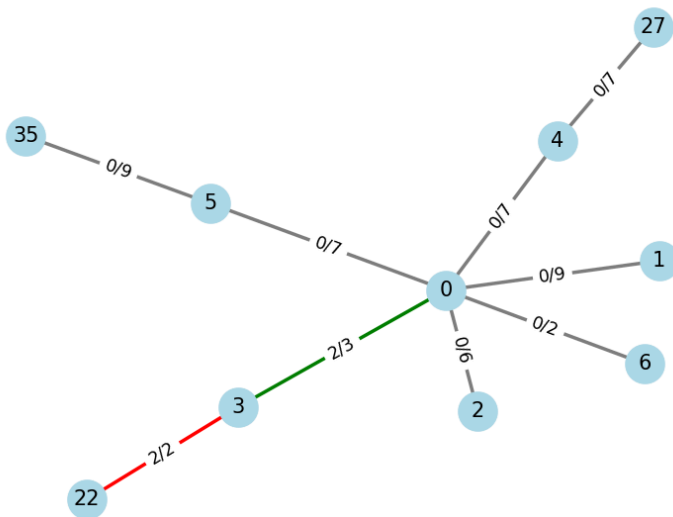


Image 2:

```
[Fractal Graph] Source: 0, Sink: 22  
Max Flow: 2  
Normalized Flow: 2.00 / 52 = 0.038  
Active Edges: 2
```

Result Comparison:

Due to the low vertex amount, the traditional graph looks similar to the fractal network, which is unfortunate and might lead to a lack of sufficient differences. Because of this, I will add another run-through with 20 vertices.

The traditional graph with its linear-like node connectivity has a few differences in flow distribution when compared to the fractal network. The 10 vertices simulation does not show this perfectly, but:

It is clear that the traditional graph, simply the generation of it, has fewer bottlenecks.

The graph is also generated in a more compact manner, however this could be both good and bad.

The traditional graph also uses more edges, which can be a drawback when considering the price and economical factors of these cities, “plans.”

Due to the random nature of the capacity values, this point is hard to prove consistently, but the traditional graph certainly makes better use of its capacity limits, with the potential to take other paths when some are full.

The fractal graph, with its recursive node generation, also has a few differences in flow distribution when compared to the traditional network (obviously since we’re comparing them). Unlike the traditional graph though, the amount of vertices of the fractal graph do not monumentally change the findings, due to its recursive nature. Nevertheless:

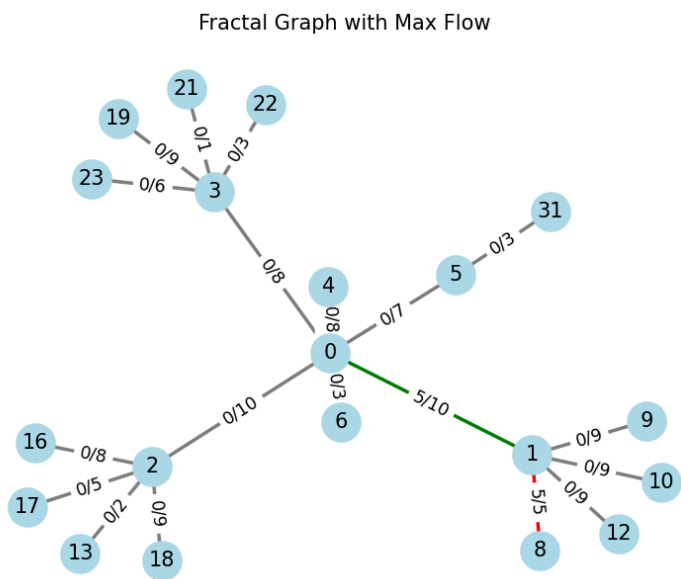
The fractal graph is less random, and more complex, however if our source was anything but node 0, i.e. the center, to reach any one branch from another would require the path to cross through node 0. In a more hypothetical, “real life scenario,” this could mean a whole shutdown of the system if something were to happen to node 0, as now we couldn't move from one leg to another.

The flow of a fractal graph is certainly handicapped by setting the source to 0, however any other source broke my program. Which brings up a good point as to how to compare the two graphs to try and set the constraints to ensure the fairness of the test.

Both graphs have their own unique characteristics that impact the flow distribution and efficiency. The traditional graph tends to have fewer bottlenecks and makes better use of the capacity restrictions. This can be both a pro and a con depending on budget constraints in a real world scenario. The fractal graph, on the other hand, has a more complex structure and can lead to monumental disaster when the central node fails. The design allows for efficient capacity usage in some areas, but it also highlights the importance of the source placement. It’s also interesting to see how each node, apart from 0, is connected to only one other.

```
[Basic Graph] Source: 0, Sink: 12
Max Flow: 2
Normalized Flow: 2.00 / 117 = 0.017
Active Edges: 14
```

Fractal Graph Generation (20 verts):



Stats Image:

```
[Fractal Graph] Source: 0, Sink: 8
Max Flow: 5
Normalized Flow: 5.00 / 124 = 0.040
Active Edges: 2
```

Conclusion:

I most definitely feel as though with this project I've bitten off more than I could chew and thus the quality/accuracy of the project feels lackluster. The graph generation and first half of the project up to the checkpoint went nicely. After being recommended to start focusing on the network flow aspect though, it began to go downhill.

I learned about a whole new section of graph theory. Additionally I had to translate it to code which was not easy. The code is very messy, with repeating function names and potentially even two functions to do the same task but it's too late to fix. Which on a separate note, is why I love to try and finish all my coding assignments in one sitting as I struggle to come back to a half finished program and remember how I was thinking during that time. Regardless, I would love to clean it up given time, alas, said time I do not have

Despite the challenges faced, I cannot help but to smile and feel proud after each graph generation. I managed to create and compare two very different types of graphs under a network flow analysis. Even with the imperfections and bugs/logic errors the process taught me a fair amount. Python, Matplotlib, python interpreters, NetworkX, various network algorithms, graph theory principles, managing complexities, adjusting scores, and pushing through setbacks and adapting to changes. In the end, the project/assignment gave me a deeper appreciation for the ancient cities, even modern ones, but I've always been attracted to ancient history so I may be a little biased.

The structure and urban planning of these cities never came across my mind, and how much thought goes into planning them. If I had more time and skill, I'd work on making the code more efficient, polish the network flow aspect of it, and perhaps add back the user input for more custom simulations.

In a more technical manner, for a conclusion I want to summarize how the traditional graph demonstrated greater flexibility with a variety of potential paths, fewer bottlenecks, and better overall capacity utilization. This resembles the growth and adaptability of organically grown cities. However, in hindsight, I should have attempted to create a grid-like graph as the aim was to compare city structures and modern cities follow the grid-like pattern.

In contrast though, the fractal graphs recursive design created a more centralized network. While it certainly looks better, the design is prone to error. A disruption of the central node would monumentally reduce the path count especially between branches which stem from the central node. The flow efficiency was also constrained with this bottleneck, "format," as paths funnel through the one central channel everywhere else.

Both graphs ensured full connectivity, but their topologies impacted how they handled the source-sink flow. As mentioned earlier, a personal hypothesis of mine after this project is that with guaranteed node success, a fractal network would prove superior, generally. However this still doesn't solve congestion and capacity issues, but perhaps that issue could be solved in a 3d space with nodes going up/down, or another fractal algorithm. If node success is not guaranteed though, I believe it'd be best to stick with a traditional graph.

Perhaps modern day engineers don't use the idea of fractal cities for a reason. Sadly, as interesting and as cool as they are, the future seems to be grid-locked.

Works Cited

Author links open overlay panelTimoteo Carletti, AbstractIn this paper we define a new class of weighted complex networks sharing several properties with fractal sets, Boccaletti, S., Fortunato, S., Barabási, A.-L., Guan, J., Erdős, P., Albert, R., Watts, D. J., Yook, S. H., Barrat, A., Jung, S., Dorogovtsev, S. N., Ravasz, E., Zhang, Z. Z., Zhang, Z., & Song, C. (2010, January 20). *Weighted Fractal Networks*. Physica A: Statistical Mechanics and its Applications. <https://www.sciencedirect.com/science/article/abs/pii/S0378437110000518>

Flow network: Brilliant math & science wiki. Brilliant. (n.d.).
<https://brilliant.org/wiki/flow-network/#:~:text=A%20Flow%20network%20is%20a,of%20routes%20with%20limited%20capacity>.

Fractal applications. Fractal Foundation Online Course - Chapter 12 - FRACTAL APPLICATIONS. (n.d.). <https://fractalfoundation.org/OFC/OFC-12-3.html>

(PDF) box-covering algorithm for fractal dimension of weighted networks. (n.d.).
https://www.researchgate.net/publication/258043389_Box-covering_algorithm_for_fractal_dimension_of_weighted_networks

UCF. (n.d.-b). <https://www.cs.ucf.edu/~dmarino/progcontests/cop4516/notes/NetworkFlow.pdf>

W3schools.com. W3Schools Online Web Tutorials. (n.d.).
https://www.w3schools.com/dsa/dsa_algo_graphs_edmondskarp.php#:~:text=The%20Edmonds%20DKarp%20algorithm%20works,as%20possible%20through%20that%20path.